

Chapter 4

The Processor

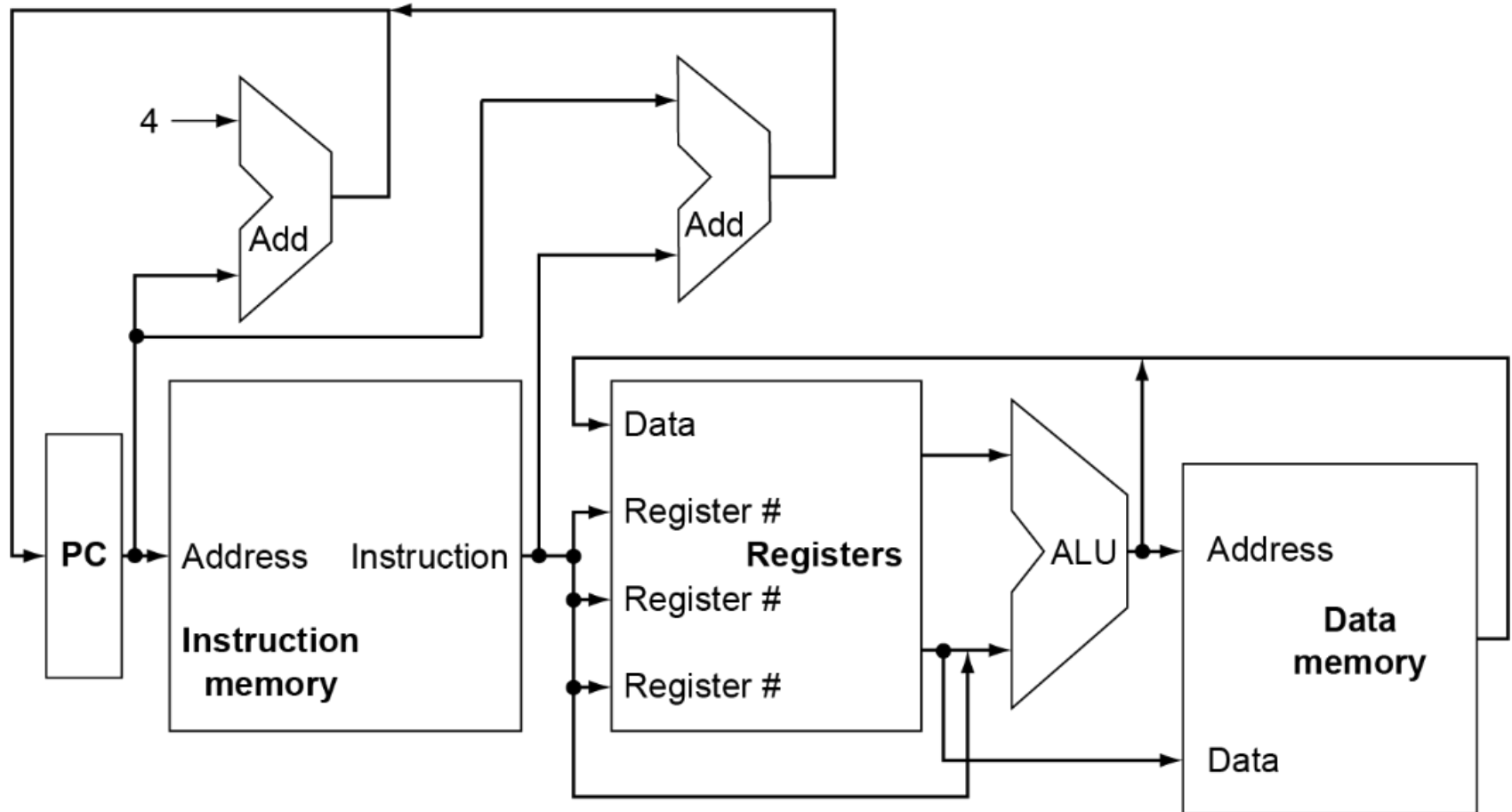
Introduction

- CPU performance factors
 - Instruction count
 - Determined by ISA and compiler
 - CPI and Cycle time
 - Determined by CPU hardware
- We will examine two RISC-V implementations
 - ✓ ■ A simplified version
 - ✓ ■ A more realistic pipelined version
- Simple subset, shows most aspects
 - ✓ ■ Memory reference: ld, sd
 - ✓ ■ Arithmetic/logical: add, sub, and, or
 - ✓ ■ Control transfer: beq

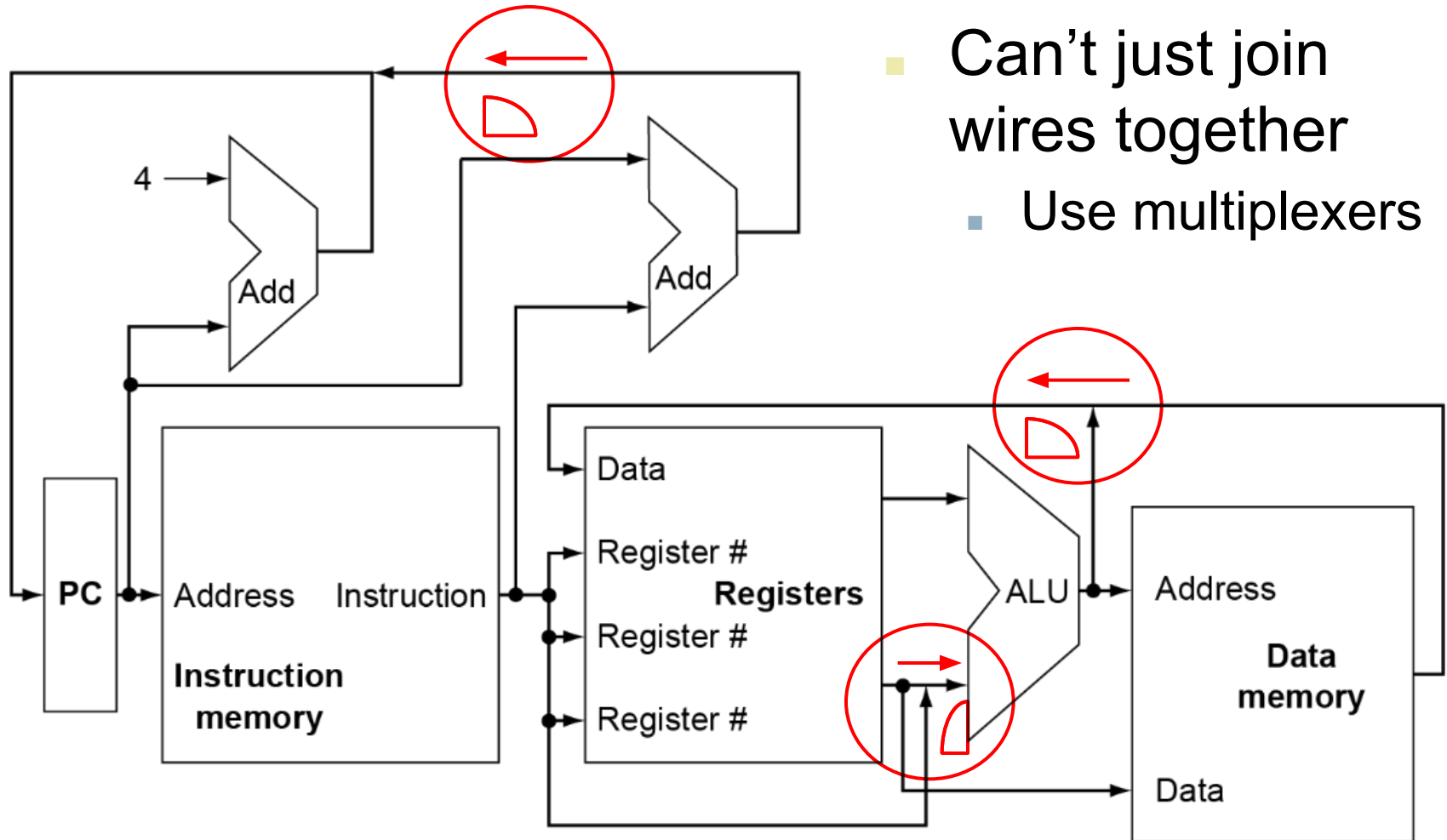
Instruction Execution

- ✓ ■ PC → instruction memory, fetch instruction
- ✓ ■ Register numbers → register file, read registers
- ✓ ■ Depending on instruction class
 - Use ALU to calculate
 - Arithmetic result ✓
 - Memory address for load/store ✓
 - Branch comparison ✓
 - Access data memory for load/store
 - $PC \leftarrow \text{target address or } PC + 4$

CPU Overview

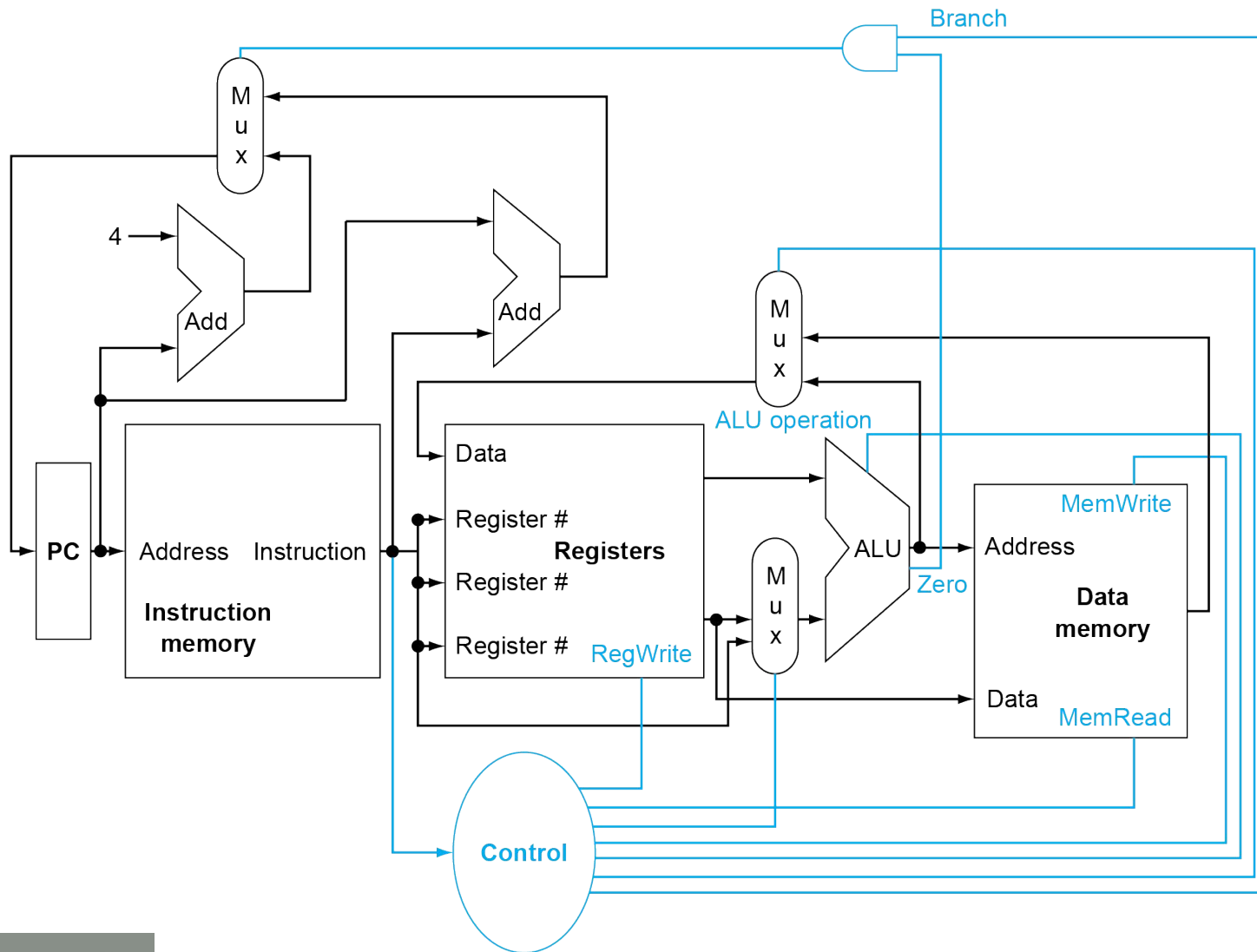


Multiplexers



- Can't just join wires together
 - Use multiplexers

Control



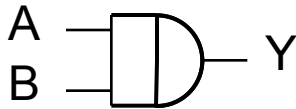
Logic Design Basics

- Information encoded in binary
 - Low voltage = 0, High voltage = 1
 - One wire per bit
 - Multi-bit data encoded on multi-wire buses
- Combinational element
 - Operate on data
 - Output is a function of input
- State (sequential) elements
 - Store information

Combinational Elements

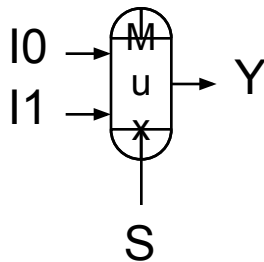
- AND-gate

- $Y = A \& B$



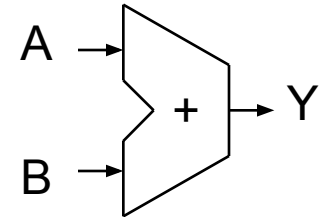
- Multiplexer

- $Y = S ? I1 : I0$



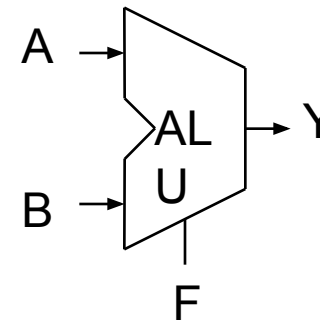
- Adder

- $Y = A + B$



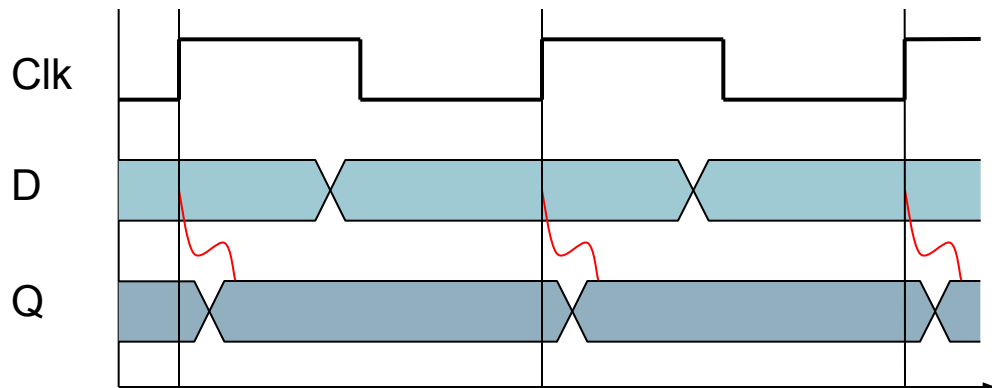
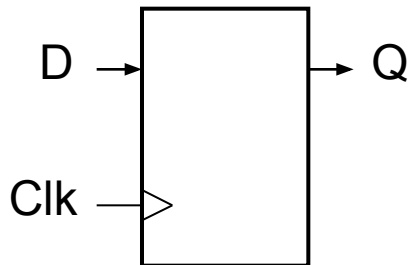
- Arithmetic/Logic Unit

- $Y = F(A, B)$



Sequential Elements

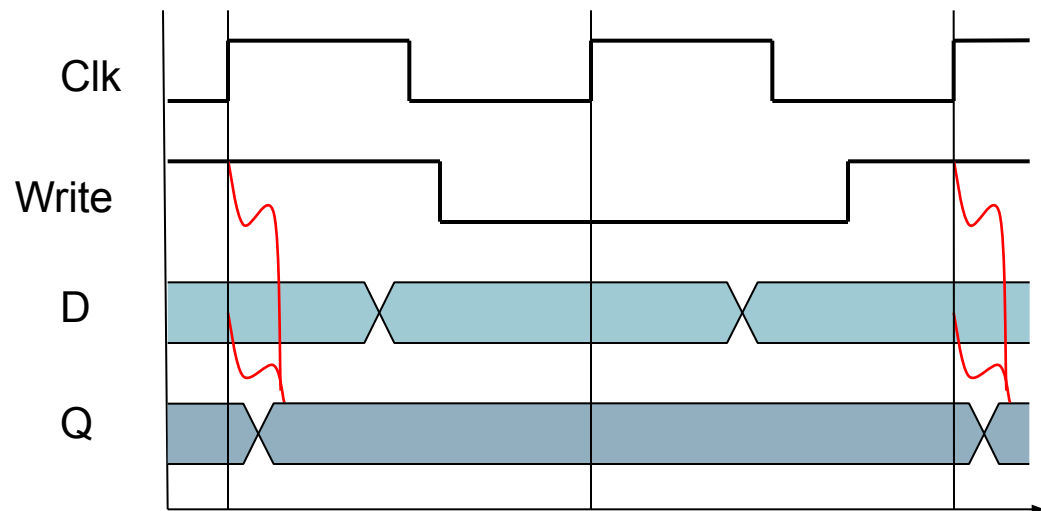
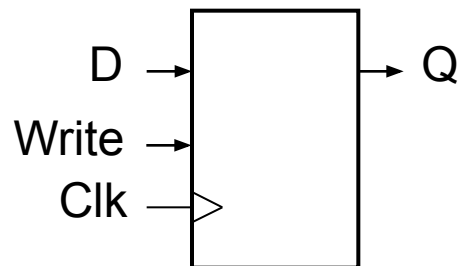
- Register: stores data in a circuit
 - Uses a clock signal to determine when to update the stored value
 - Edge-triggered: update when Clk changes from 0 to 1



Sequential Elements

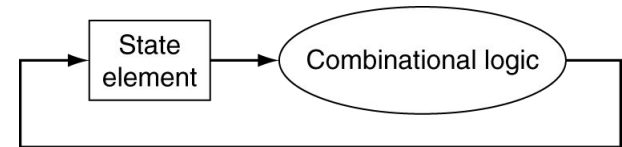
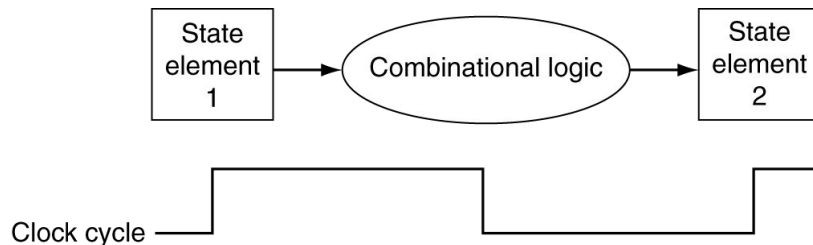
Register with write control

- Only updates on clock edge when write control input is 1
- Used when stored value is required later



Clocking Methodology

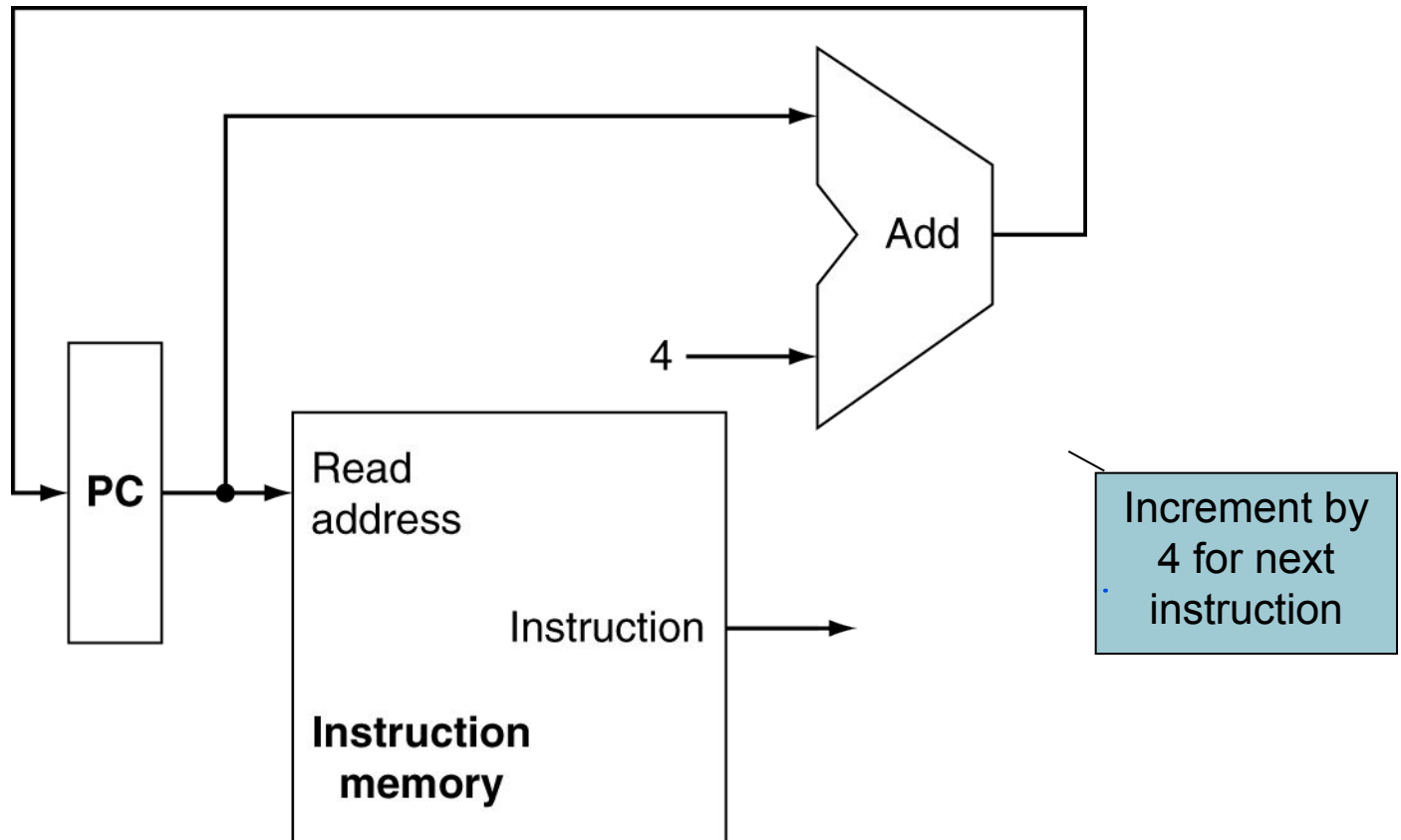
- Combinational logic transforms data during clock cycles
 - Between clock edges
 - Input from state elements, output to state element
 - Longest delay determines clock period



Building a Datapath

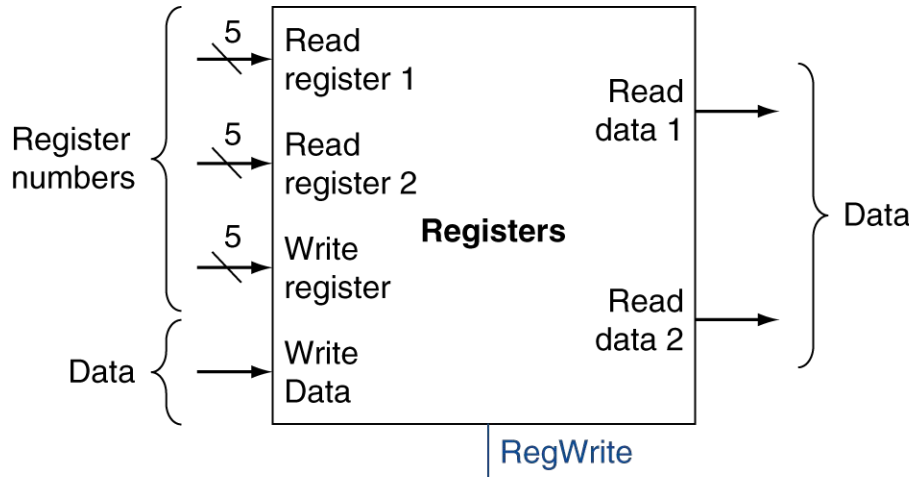
- Datapath
 - Elements that process data and addresses in the CPU
 - Registers, ALUs, mux's, memories, ...
- We will build a RISC-V datapath incrementally
 - Refining the overview design

Instruction Fetch

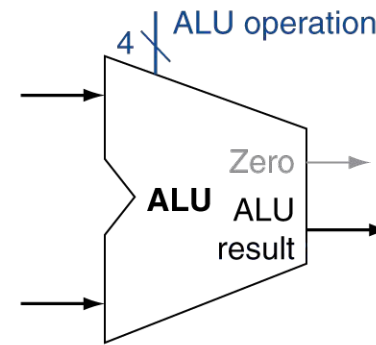


R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result



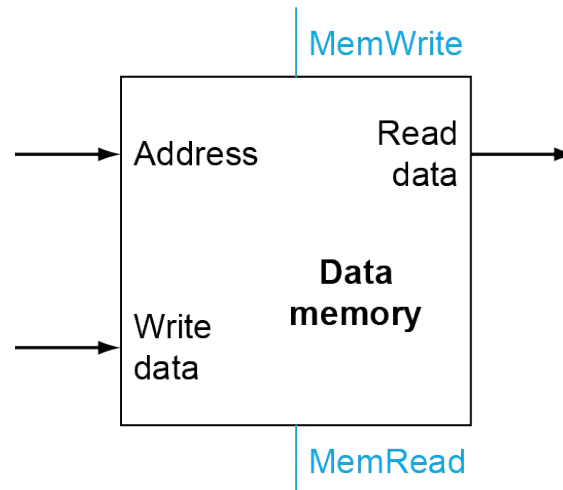
a. Registers



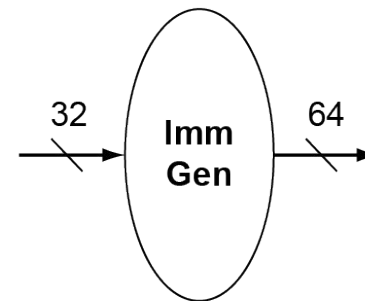
b. ALU

Load/Store Instructions

- Read register operands
- Calculate address using 12-bit offset
 - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit

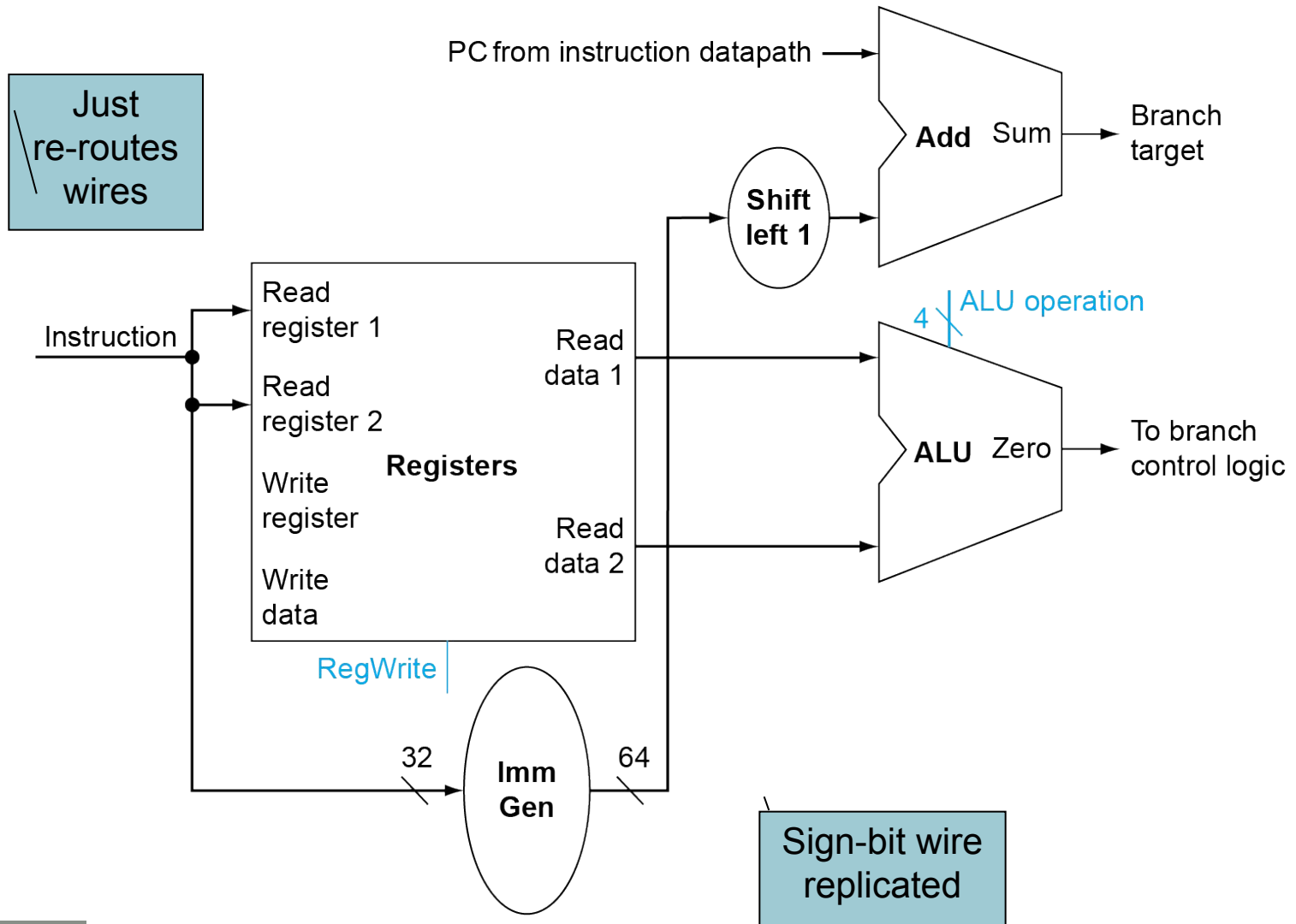


b. Immediate generation unit

Branch Instructions

- Read register operands
- Compare operands
 - Use ALU, subtract and check Zero output
- Calculate target address
 - Sign-extend displacement
 - Shift left 1 place (halfword displacement)
 - Add to PC value

Branch Instructions



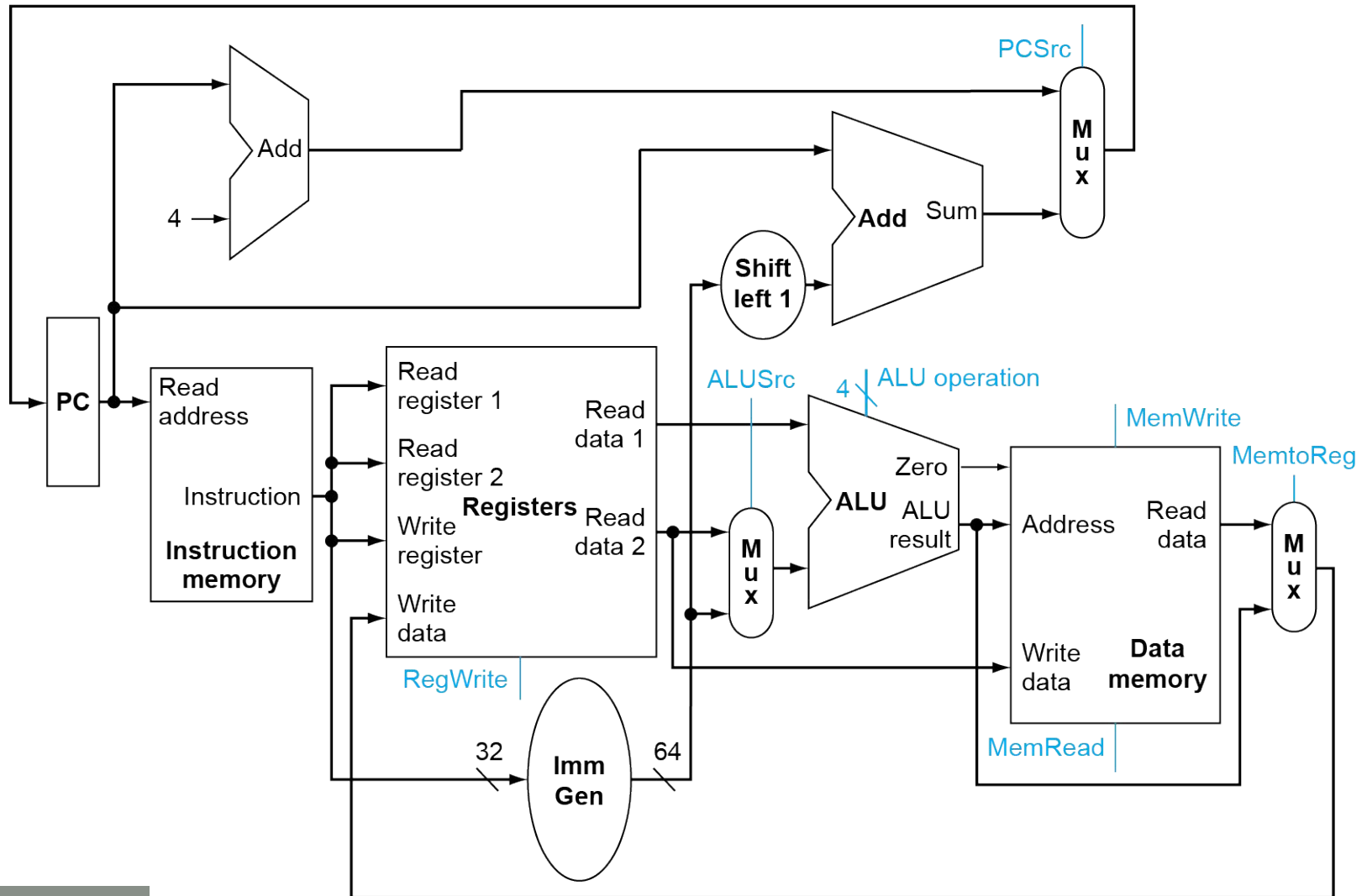
Composing the Elements

- First-cut data path does an instruction in one clock cycle
 - Each datapath element can only do one function at a time
 - Hence, we need separate instruction and data memories
- Use multiplexers where alternate data sources are used for different instructions

MK



Full Datapath



ALU Control

- ALU used for
 - Load/Store: F = add
 - Branch: F = subtract
 - R-type: F depends on opcode

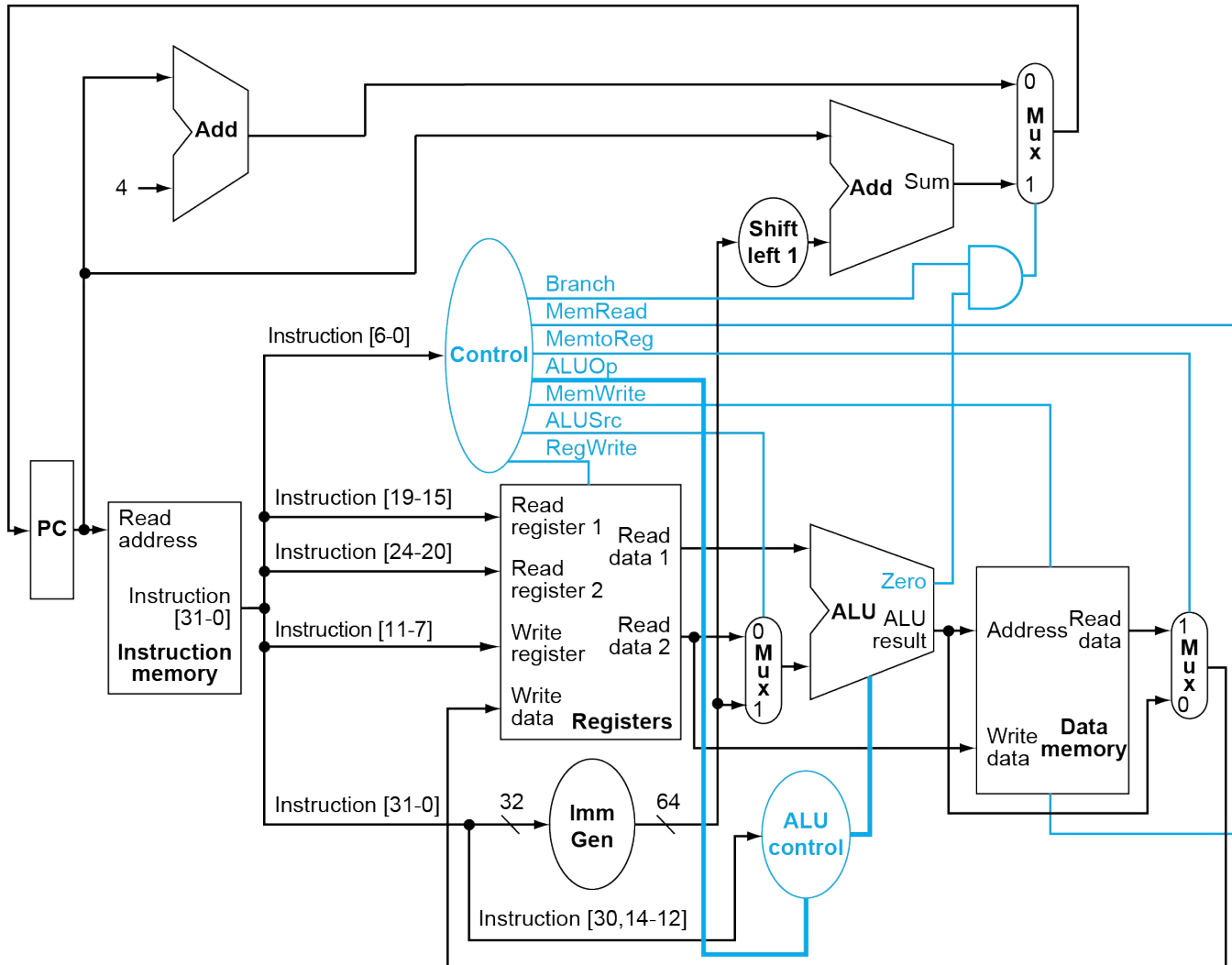
ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract

ALU Control

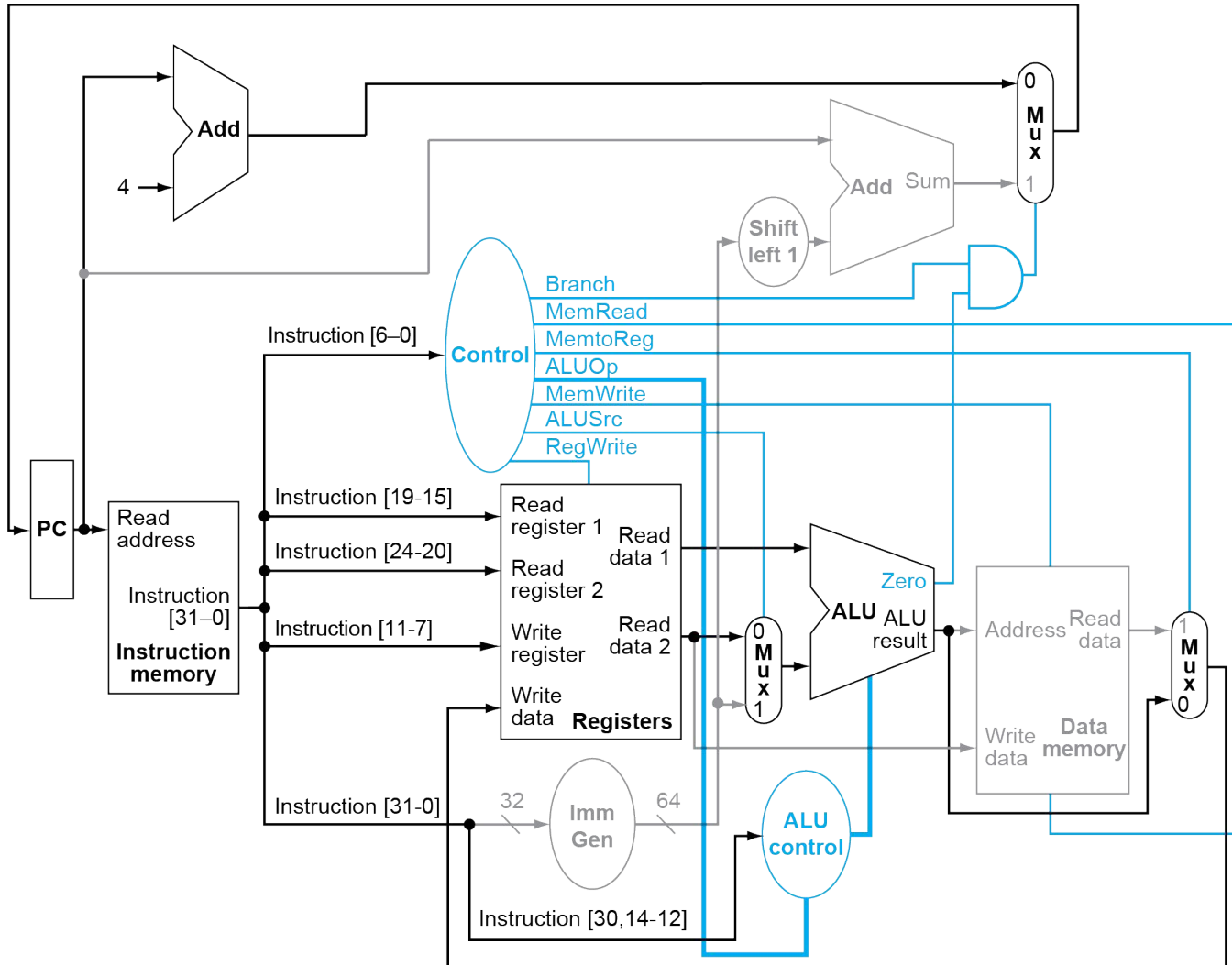
- Assume 2-bit ALUOp derived from opcode
 - Combinational logic derives ALU control

opcode	ALUOp	Operation	ALU function	ALU control
ld	00	load register	add	0010
sd	00	store register	add	0010
beq	01	branch on equal	subtract	0110
R-type	10	add	add	0010
		subtract	subtract	0110
		AND	AND	0000
		OR	OR	0001

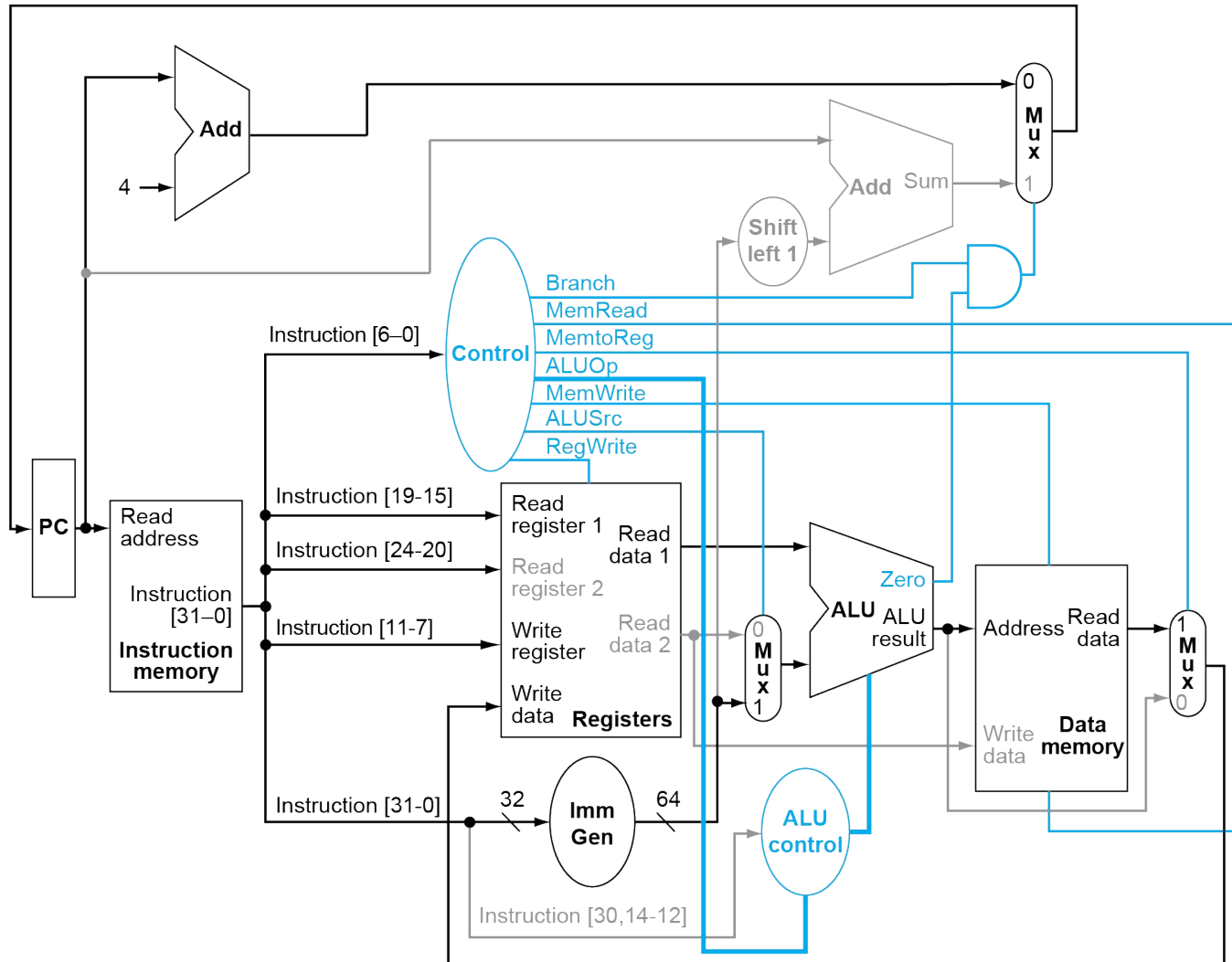
Datapath With Control



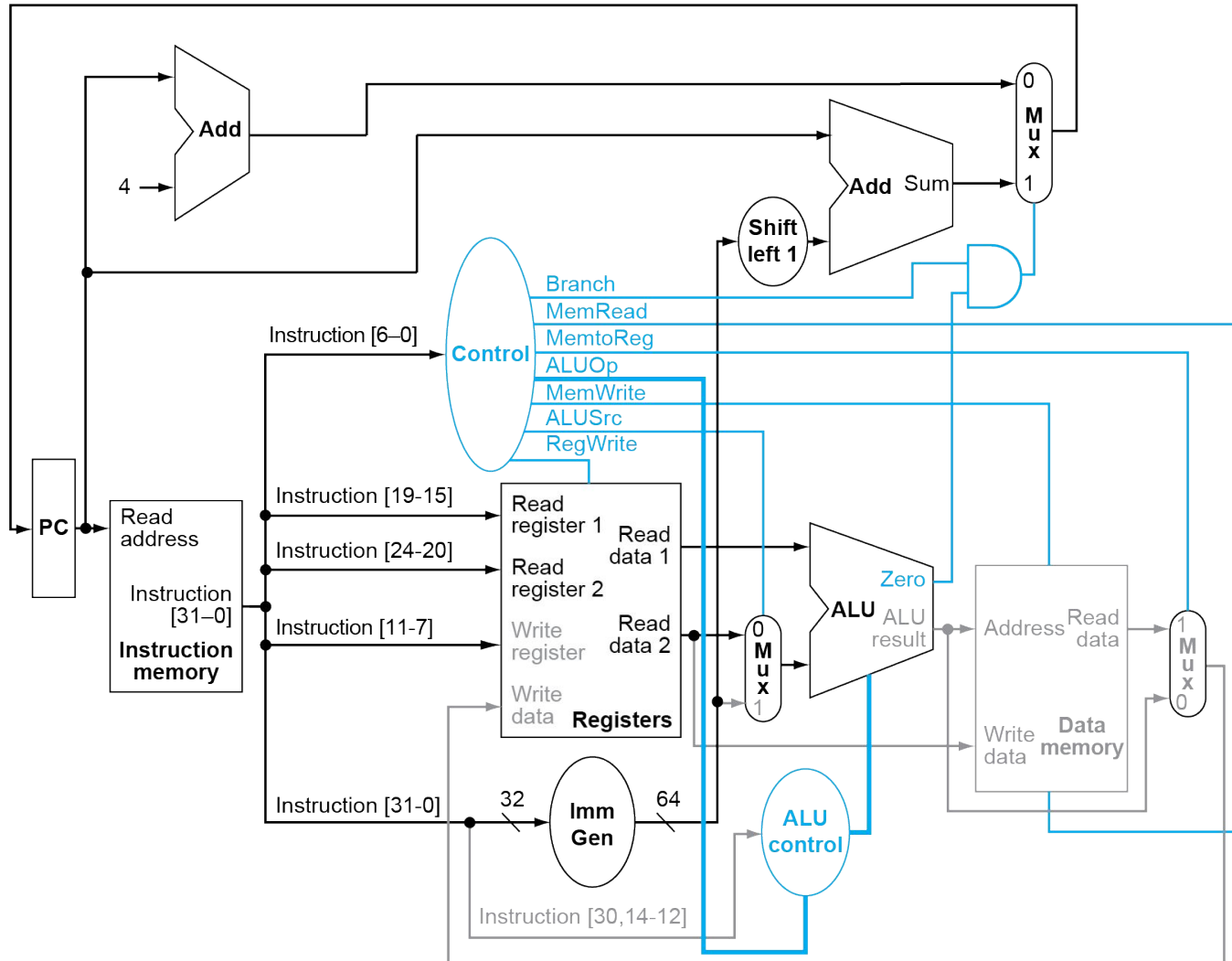
R-Type Instruction



Load Instruction



Branch-on-Equal Instruction

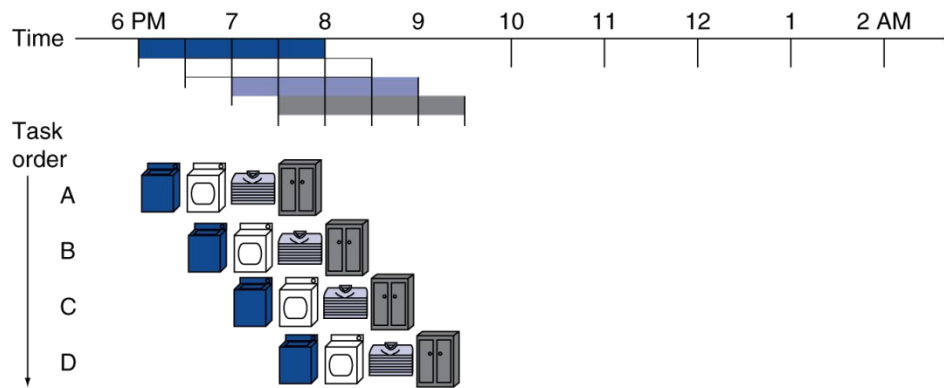
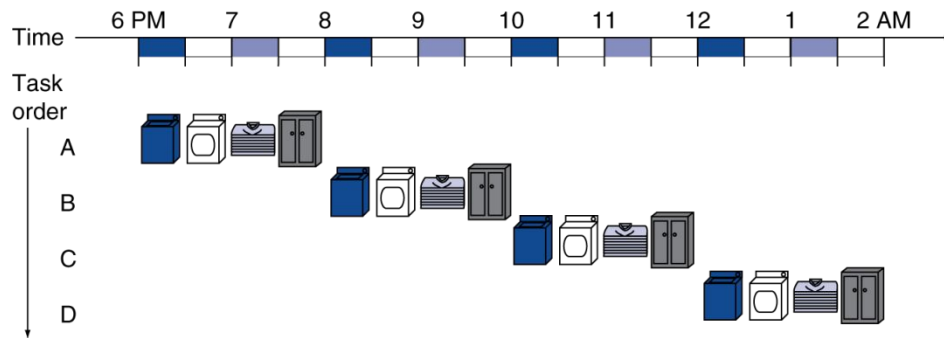


Performance Issues

- Longest delay determines clock period
 - Critical path: load instruction
 - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
 - Making the common case fast
- We will improve performance by pipelining

Pipelining Analogy

- Pipelined laundry: overlapping execution
 - Parallelism improves performance



RISC-V Pipeline

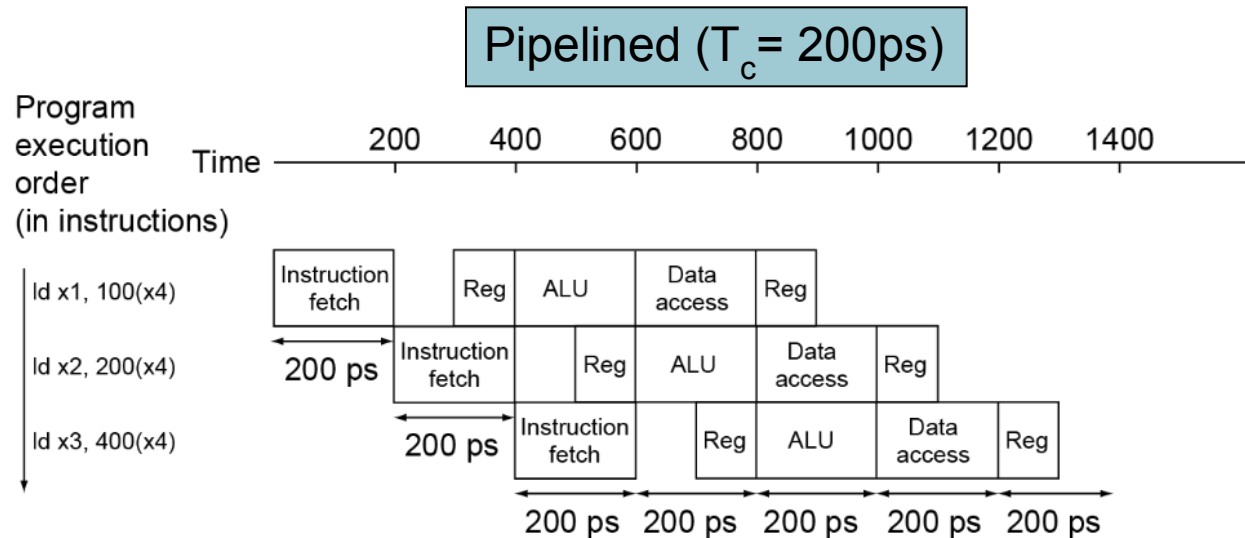
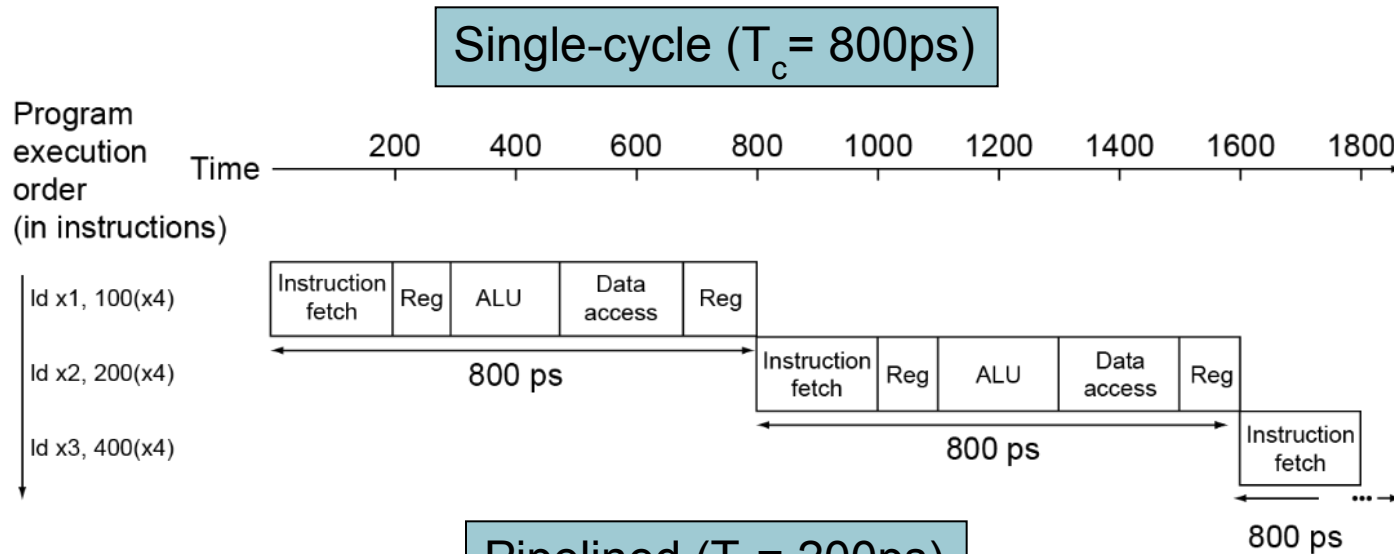
- Five stages, one step per stage
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register read
 3. EX: Execute operation or calculate address
 4. MEM: Access memory operand
 5. WB: Write result back to register

Pipeline Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
ld	200ps	100 ps	200ps	200ps	100 ps	800ps
sd	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance



Pipelining and ISA Design

- RISC-V ISA designed for pipelining
 - All instructions are 32-bits
 - Easier to fetch and decode in one cycle
 - c.f. x86: 1- to 17-byte instructions
 - Few and regular instruction formats
 - Can decode and read registers in one step
 - Load/store addressing
 - Can calculate address in 3rd stage, access memory in 4th stage

Hazards

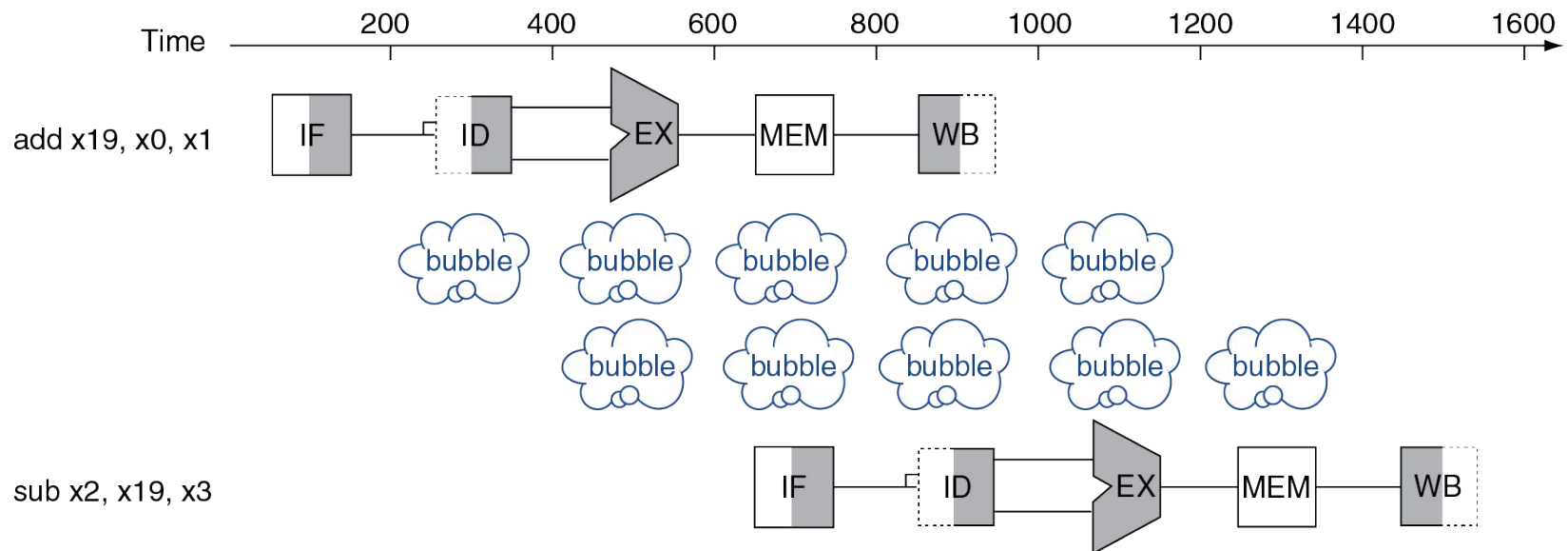
- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
 - A required resource is busy
- Data hazard
 - Need to wait for previous instruction to complete its data read/write
- Control hazard
 - Deciding on control action depends on previous instruction

Structural Hazards

- Conflict for use of a resource
- In RISC-V pipeline with a single memory
 - Load/store requires data access
 - Instruction fetch would have to *stall* for that cycle
 - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
 - Or separate instruction/data caches

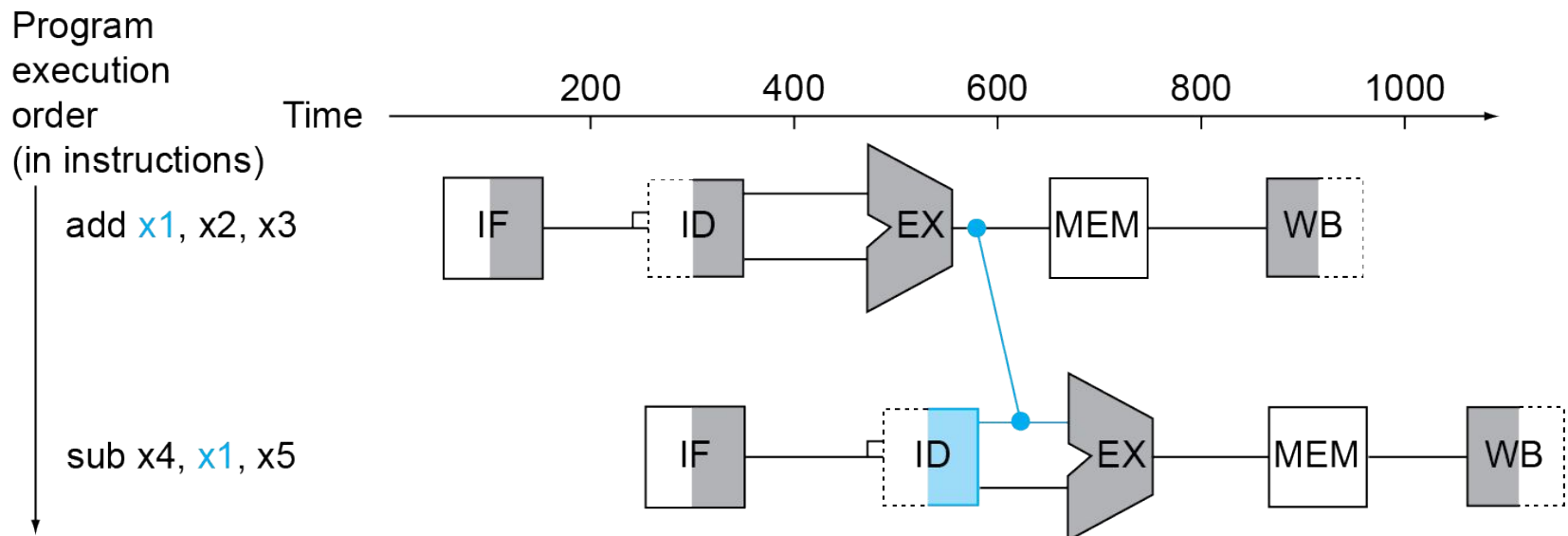
Data Hazards

- An instruction depends on completion of data access by a previous instruction
 - add **x19**, x0, x1
sub x2, **x19**, x3



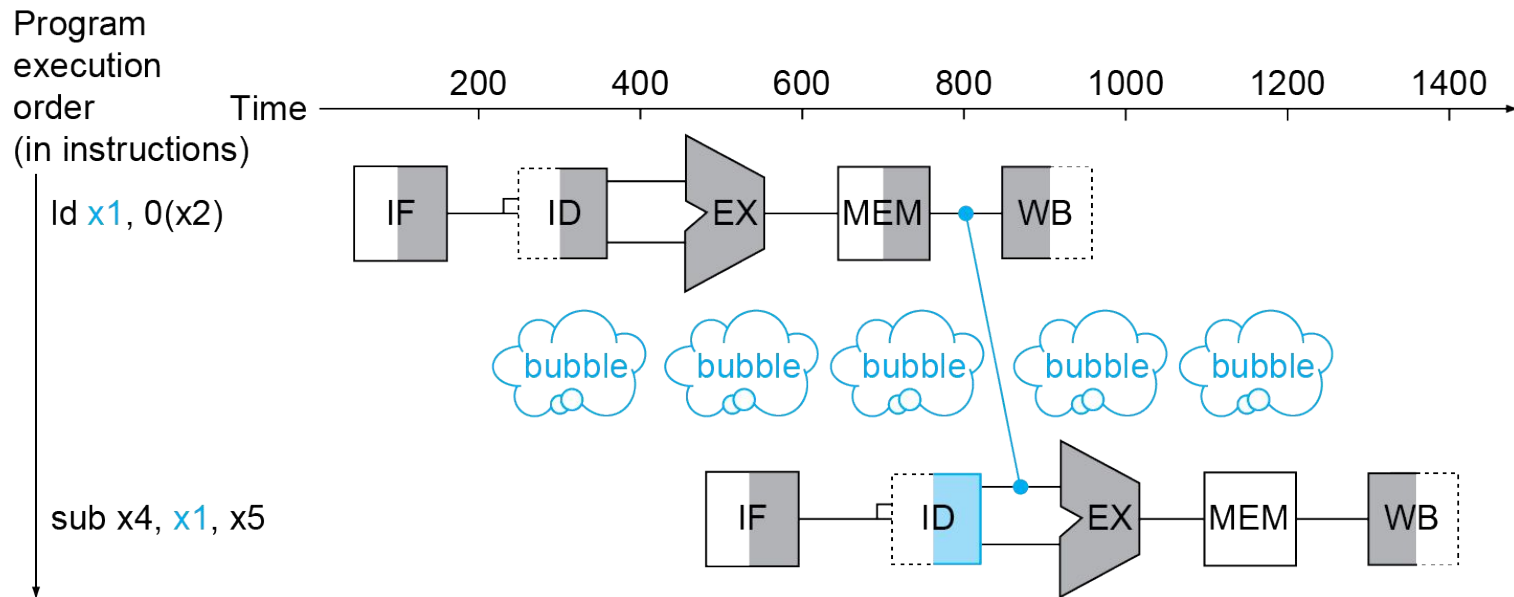
Forwarding (aka Bypassing)

- Use result when it is computed
 - Don't wait for it to be stored in a register
 - Requires extra connections in the datapath



Load-Use Data Hazard

- Can't always avoid stalls by forwarding
 - If value not computed when needed
 - Can't forward backward in time!



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for $a = b + e$; $c = b + f$;

stall

stall

```
ld    x1, 0(x0)
ld    x2, 8(x0)
add   x3, x1, x2
sd    x3, 24(x0)
ld    x4, 16(x0)
add   x5, x1, x4
sd    x5, 32(x0)
```

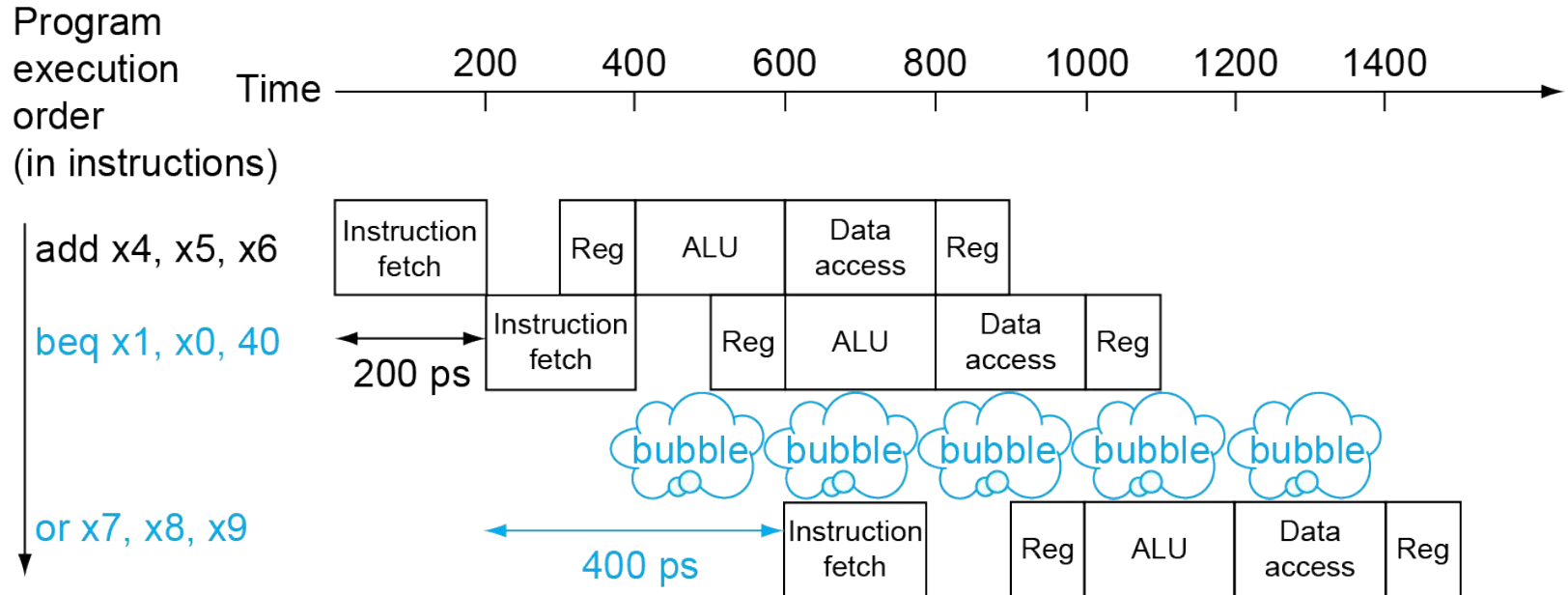
13 cycles

```
ld    x1, 0(x0)
ld    x2, 8(x0)
ld    x4, 16(x0)
add   x3, x1, x2
sd    x3, 24(x0)
add   x5, x1, x4
sd    x5, 32(x0)
```

11 cycles

Stall on Branch

- Wait until branch outcome determined before fetching next instruction



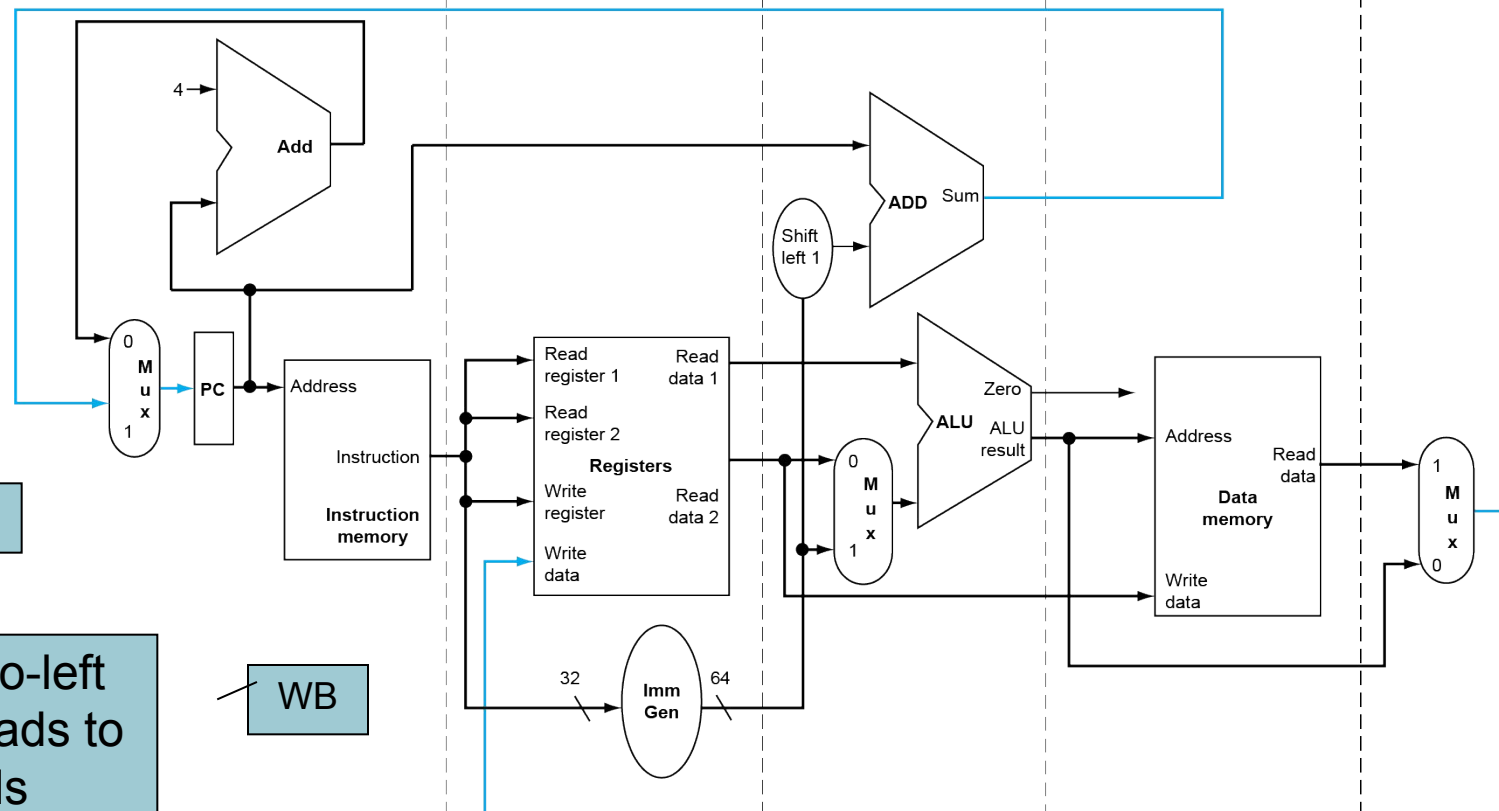
RISC-V Pipelined Datapath

IF: Instruction fetch

ID: Instruction decode/
register file readEX: Execute/
address calculation

MEM: Memory access

WB: Write back



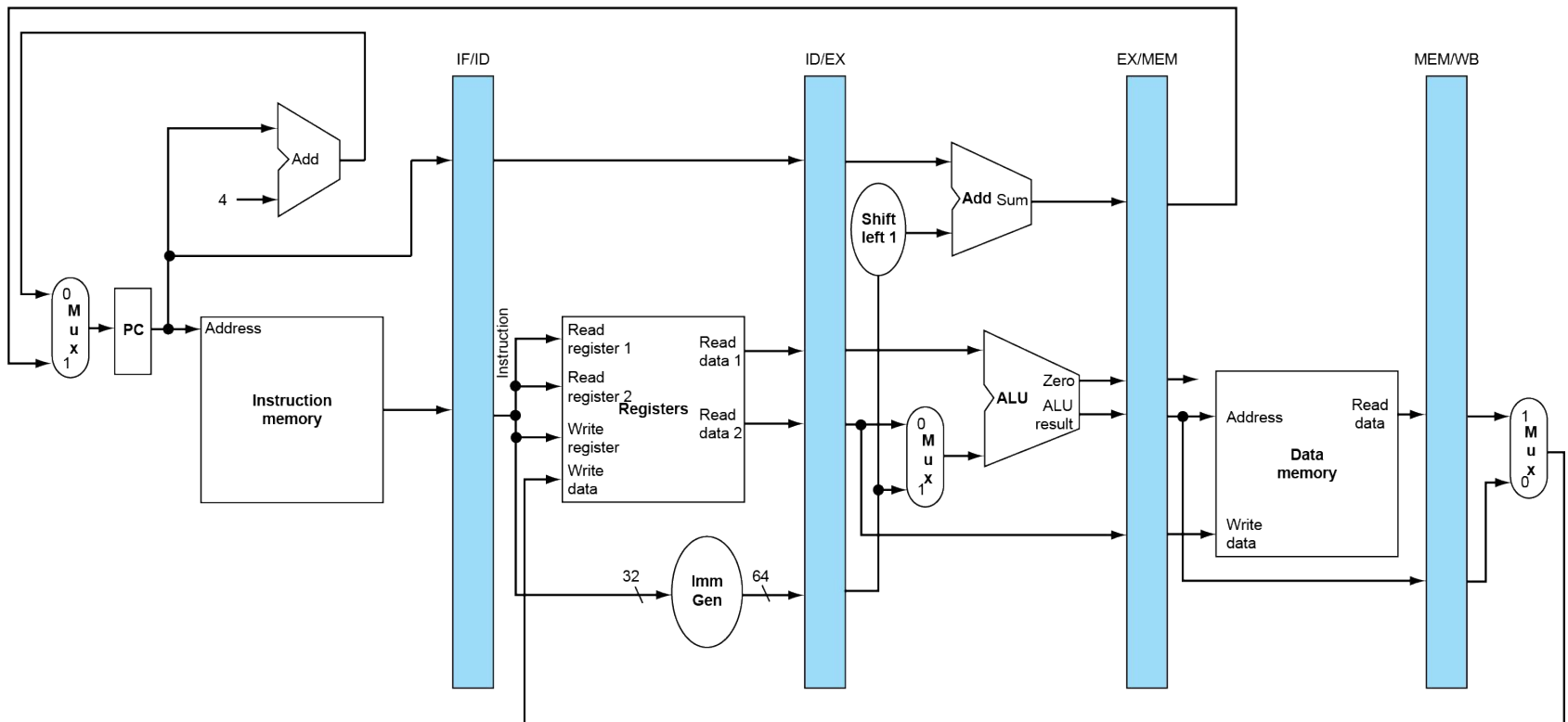
MEM

WB

Right-to-left
flow leads to
hazards

Pipeline registers

- Need registers between stages
 - To hold information produced in previous cycle



Pipeline Summary

The BIG Picture

- Pipelining improves performance by increasing instruction throughput
 - Executes multiple instructions in parallel
 - Each instruction has the same latency
- Subject to hazards
 - Structure, data, control
- Instruction set design affects complexity of pipeline implementation